

PENETRATION TESTING

„Bash & Python Scripting“



Bash

- Intro
- Variablen
- Argumente
- Input und Output
- Bedingte Anweisungen
- Boolsche Anweisungen
- Schleifen
- Funktionen

Python

- Variablen und Typen
- Listen und Dictionaries
- Schleifen & Bedingte Anweisungen
- Nutzereingaben
- Dateien und Funktionen
- Module und Webanfragen
- Netzwerk Sockets

- GNU Bourne-Again Shell (Bash)
- Einsatz zur Automatisierung von ermüdenden, sich oft wiederholenden Tasks
- Ein Bash-Skript ist eine Aneinanderreihung von Kommandos, welche nacheinander so ausgeführt werden, wie wenn man sie in die Konsole eintippt.
- Bash-Skripte haben die Dateiendung `.sh` und benötigen ausführbare Rechte, welche über `$chmod +x scriptname.sh` vergeben werden können.
- Beginn des Bash-Skripts: **`#!/bin/bash`**
- Ausführen des Skripts über: **`./<scriptname>.sh`**

```
kali@kali:~$ ./my_test.sh  
Hello World!
```

- Variablen sind benannter Speicherort in dem temporär während der Laufzeit des Skripts ein Wert gespeichert ist.
- Variablen in Bash sind case-sensitive.
- Die Deklaration erfolgt im einfachsten Fall über die Zuordnung: ***name=value***
- Bash behandelt alle Variablen zunächst als String und unterscheidet erst im Kontext ob es sich um Datentypen wie Int, Real, String, etc. handelt
- Beim Bash-Skripten muss der Unterschied zwischen Single und Double-Quotes beachtet werden z.B. für Textsubstitution

```
kali@kali:~$ firstword=Hello
kali@kali:~$ secondword=there!
kali@kali:~$ echo $firstword $secondword
Hello there!
```

```
kali@kali:~$ greeting=Hello there!
there!: command not found

kali@kali:~$ greeting="Hello there\!"
kali@kali:~$ echo $greeting
Hello there!
```

BEFEHL-SUBSTITUTION

- Mithilfe des Backquotes („`“) können Befehle oder Programmnamen in Variablen gespeichert werden: ***name="`command`"***
- Alternativ: ***name=\$(command)***
- Wenn eine Variable bspw. mittels ***echo*** aufgelöst wird, wird der Ausgabewert des Befehls verwendet.
- Das Ausführen des Befehls passiert allerdings schon beim Setzen der Variable.

```
kali@kali:~$ user="`whoami`"  
kali@kali:~$ echo $user  
kali
```

```
kali@kali:~$ user2=$(whoami)  
kali@kali:~$ echo $user2  
kali
```

- In Bash werden nur als Integer, nicht aber als Gleitkommazahlen verarbeitet.
- Spezielle Konvention für die Nutzung mathematischer Operationen in Bash
- Wird eine mathematische Operation an einem String ausgeführt der keine Zahl repräsentiert, wird dieser Variable der Wert null vergeben.
- Für mathematische Operationen kann auch der Befehl **let** verwendet werden.

```
kali@kali:~$ num1="7"
kali@kali:~$ num2=3
kali@kali:~$ echo $((num1+num2))
10
```

```
kali@kali:~$ num1="zwei"
kali@kali:~$ num2=2
kali@kali:~$ echo $((num1*num2))
0
```

- Genauso wie bei Befehlen können Skripten Argumente übergeben werden, indem diese hinter dem Skriptaufruf angefügt werden.
- Argumenten können Bezeichnungen bestehend aus einem Bindestrich oder einem doppelten Bindestrich und dem Argumentnamen zugewiesen werden.

```
kali@kali:~$ cat my_test.sh
#!/bin/bash

echo "This script has $# arguments"

kali@kali:~$ ./my_test.sh
This script has 0 arguments

kali@kali:~$ ./my_test.sh hi
This script has 1 arguments

kali@kali:~$ ./my_test.sh hi ho ha
This script has 3 arguments
```

Variablenname	Beschreibung
\$0	Name des Bashskripts
\$1 - \$9	Die ersten 9 Argumente des Bash-Skripts
\$#	Anzahl an Argumenten
\$@	Alle Argumente
\$?	Exit-Status des letzten Runs
\$\$	Prozess-ID
\$User / \$UID	Name des Nutzers / User-ID
\$HOSTNAME	Name des Systems auf welchem das Skript läuft
\$RANDOM	Zufallszahl
\$LINENO	Aktuelle Zeilennummer

INPUT & OUTPUT

NUTZEREINGABEN

- Mithilfe des Befehls **read** kann ein Nutzer im Bash-Skript zu einer Eingabe aufgefordert und der Wert in einer Variable gespeichert werden.
- Wichtige Optionen:
 - p: Angabe eines Prompts
 - s: stille Nutzereingabe

```
kali@kali:~$ cat my_test.sh
#!/bin/bash

# Prompt the user for credentials
read -p 'Username: ' username
read -sp 'Password: ' password
echo "Credentials: " $username " and " $password

kali@kali:~$ ./my_test.sh
Username: user1
Password: Credentials:  user1  and  password1234
```

INPUT & OUTPUT REDIRECTION

- Auch in Kombination mit Bash Scripting können die bekannten Operatoren > und < verwendet werden.
- Input kann in Form von Dateien z.B. Textdateien an ein Shell-Skript übergeben werden (Schlüsselwort read).
- Output kann in eine Textdatei geschrieben werden.

```
kali@kali:~$ cat my_test.sh
#!/bin/bash

#!/bin/bash
read line
echo $line

kali@kali:~$ ./my_test.sh < text.txt
Text aus einer Datei
```

```
kali@kali:~$ touch newfile.txt

kali@kali:~$ echo Dies ist ein neues File >> newfile.txt

kali@kali:~$ cat newfile.txt
Dies ist ein neues File
```

INPUT & OUTPUT

DATEIMANAGEMENT

Befehl	Beschreibung
ls	Auflisten von Dateien in einem Verzeichnis
rm	Löschen einer Datei
touch	Erstellen eines leeren Files
head	Ausgeben der ersten Zeilen eines Files
tail	Ausgeben der letzten Zeilen eines Files
mkdir	Anlegen eines neuen Verzeichnisses
cp	Kopieren des Dateiinhalts
cd	Navigieren durch das Verzeichnissystem

BEDINGTE ANWEISUNGEN

IF-ANWEISUNGEN

- Die if-Anweisung unterliegt in Bash einer sehr spezifischen Syntax:

```
if [ <Bedingung1> ]
then
    <Aktion1>
elif [ <Bedingung2> ]
then
    <Aktion2>
else
    <Aktion>
fi
```

```
1 #!/bin/bash
2
3 read -p "Enter your age:" age
4
5 if [ $age -lt 18 ]
6 then
7     echo "Ask your parents for permission"
8 fi
9
```

Operatoren	Beschreibung
!Ausdruck	Ausdruck ist false
-eq / ==	equal
-ne	not equal
-gt	greater than
-lt	less than
-le	less than or equal
-ge	Greater than or equal

- Boolesche Operatoren werden u. a. In Kommandolisten eingesetzt.
- Mithilfe des Pipe-Operators („|“) wird der Output eines Kommandos an den Input eines anderen übergeben.
- Der vorherige Befehl wurde erfolgreich ausgeführt wenn True oder „0“ zurückgegeben wird.
- Der vorherige Befehl wurde nicht erfolgreich ausgeführt wenn False oder ein Wert ungleich null zurückgegeben wird.
- Boolesche Verknüpfungen:
 - AND (&&): die nachfolgenden Befehle werden erst ausgeführt, wenn die vorhergehenden erfolgreich ausgeführt wurden.
 - OR (||): die nachfolgenden Befehle werden nur dann ausgeführt, wenn der vorhergehende Befehl nicht erfolgreich ausgeführt wurde.

- For-Schleife:

```
for var-name in <list>
do
    <action to perform>
done
```

- For-Schleifen können in Bash-Skripts auch als Einzeiler (mit dem Semikolon als Seperator) geschrieben werden:

\$for ip in \$(seq 1 10); do echo 10.11.1.\$ip; done

- While-Schleife:

```
while [ <some test> ]
do
    <perform an action>
done
```

- Die ausgegebenen IP-Adressen können in Kombination mit dem Befehl ***ping*** oder ***nmap*** verwendet werden.

- 2 Formate von Funktionen:

```
function function_name {  
  commands ...  
}
```

```
function_name () {  
  commands ...  
}
```

- Definierte Funktionen werden beim Durchlauf des Programms zunächst registriert, wobei der Code in geschweiften Klammern nicht weiter betrachtet wird.
- An Funktionen können beim Aufruf Argumente übergeben werden: ***function_name \$arg***
- Funktionen in Bash können keinen eigentlichen Returnwert liefern, stattdessen geben sie einen Exit Status zurück.
- Ein Returnwert kann über eine globale Variable oder eine Befehlsausführung innerhalb der Funktion simuliert werden.
- Innerhalb von Funktionen können lokale Variablen definiert werden, welche nur innerhalb der Funktion gelten (Schlüsselwort local).

AUSFÜHREN VON PYTHON SKRIPTEN

- Mithilfe der sog. Shebang und dem Python-Pfad kann dem System mitgeteilt werden, dass es sich bei dem Skript um ein Python-Skript handelt:

```
#!/usr/bin/python
```

- Ausführen von Python-Skripts:

1. `python examplescript.py`
2. `./examplescript.py`

```
#!/usr/bin/python  
  
print("Python Scripting")
```

```
C:\Users\steff\Desktop>.\examplescript.py  
Python Scripting
```

VARIABLEN UND TYPEN

SETZEN VON VARIABLEN

- In Python werden Variablen folgendermaßen zugewiesen:
 - Variablenname = Variablenwert
- Zum Beispiel:
 - Count = 1
 - Fruit = „banana“

```
#!/usr/bin/python  
  
Count = 1  
Fruit = "banana"  
  
print (Count)  
print (Fruit)
```

```
C:\Users\steff\Desktop>.\examplescript.py  
1  
banana
```

VARIABLEN UND TYPEN

DATENTYPEN

- Mithilfe der Funktion `type()` kann überprüft werden, welchen Datentyp einer Variable zugewiesen ist.
- Der Datentyp kann mithilfe von Typcasting geändert werden:
 - `int()`
 - `str()`
 - `float()`

```
#!/usr/bin/python

Count = 1
Fruit = "banana"

print(type(Count))
print(type(Fruit))
```

```
C:\Users\steff\Desktop>.\examplescript.py
<class 'int'>
<class 'str'>
```


VARIABLEN UND TYPEN

STRINGS

- String-Werte werden in Anführungszeichen gesetzt.
- Der String-Wert ist ein Array an Chars, was bspw. dazu verwendet werden kann, um bestimmte Teile des Strings auszuschneiden.
- Der Index in einer String-Variable kann auch über die Funktion ***index()*** bestimmt werden:

```
start = „http“
```

```
tag.index(start)
```

```
#!/usr/bin/python
```

```
tag = '<a href="https://www.google.com"></a>'
```

```
url = tag[9:30]
```

VARIABLEN UND TYPEN

BOOLEAN

- Es gibt nur die beiden boolschen Werte **True** und **False**.
- Hier muss auf Case Sensitivität geachtet werden
- Boolsche Werte werden für Bedingte Anweisungen verwendet z.B. if ... else

```
#!/usr/bin/python

isSent = True

if(isSent):
    print("Successfully sent!")
else:
    print("No message has been sent yet")
```

```
C:\Users\steff\Desktop>.\examplescript.py
Successfully sent!
```

PYTHON LISTEN

- Eine Liste ist ein Datentyp, welcher eine oder mehrere Variablen in indizierter Reihenfolge enthält.
- Listen können weitere Listen enthalten.
- Mit der Funktion ***append()*** können neue Variablen der Liste hinzugefügt und mit ***remove()*** wieder entfernt werden.
- Mit der Funktion ***index()*** kann der Index einer Variable ausgegeben werden.
- Mithilfe der Funktion ***len()*** kann die Länge einer Liste ausgegeben werden.

```
#!/usr/bin/python  
  
Colours = ["red", "blue", "green"]
```

LISTEN & DICTIONARIES

DICTIONARIES

- In Python ist ein Dictionary eine Datenstruktur, welche mehrere Key-Value-Paare enthält.
- Ein Wert (Value) wird über den zugehörigen Key referenziert
- Mithilfe der Funktion **keys()** kann eine Liste der Keys ausgegeben werden

```
#!/usr/bin/python

student1 = {
    "vorname" : "paul",
    "nachname" : "meier",
    "alter" : "24"
}

student1["studium"] = "ai"
```

SCHLEIFEN & BEDINGTE ANWEISUNGEN

SCHLEIFEN

- In Python existieren zwei Arten von Schleifen: for und while
- Die while-Schleife wird so lange durchlaufen, wie die bedingte Anweisung wahr ist.
- Die for-Schleife wird so lange ausgeführt wie spezifiziert.

```
#!/usr/bin/python

i = 0
while i < 10:
    print(i)
    i += 1
```

```
#!/usr/bin/python

for i in range(10):
    print(i)
```

BEDINGTE ANWEISUNGEN

- Bedingte Anweisungen werden dann verwendet, wenn Codeblöcke nur in bestimmten Situationen ausgeführt werden soll.
- Für bedingte Anweisungen werden **if**, **elif** und **else** verwendet.

```
#!/usr/bin/python

i = True

if i:
    print("It's True!")
else:
    print("It's False!")
```

```
#!/usr/bin/python

i = 500

if i > 200:
    print("It's over 200")
elif i > 100:
    print("It's over 100")
elif i > 50:
    print("It's over 50")
else:
    print("Less than 50")
```

- Mithilfe der Funktion ***input()*** wird ein Nutzer zur Eingabe aufgefordert.
- Nutzereingaben sind vom Typ String.

```
#!/usr/bin/python

i = input("Add some random number:")

print(i)
```

```
#!/usr/bin/python

i = input("Add some random number:")

if i > 5:
    print("Greater than 5!")
else:
    print("Equal or Less than 5!")
```


DATEIEN

- Um eine Datei zu öffnen kann der Befehl ***open()*** verwendet werden. Diesem werden der Dateiname und der Modus übergeben.
- Mögliche Modi sind:
 - Read (r)
 - Write (w)
 - Append (a)
 - Read + write (r+)
 - Read binary (rb)
 - Write binary (wb)
- Der Inhalt einer Datei kann mit ***read()*** oder ***readlines()*** ausgelesen werden
- Eine Datei kann mithilfe der Funktion ***write()*** beschrieben werden.
- Nach dem Lesen oder Schreiben kann eine Datei mit ***close()*** geschlossen werden

```
f = open("file.txt", "r")  
  
data = f.read()  
  
f.close()
```

DATEIEN UND FUNKTIONEN

FUNKTIONEN

- Als Funktion wird ein Codeblock bezeichnet, der im Skript oder auch von externen Programmen referenziert werden kann
- Funktionen müssen im Skript vor dem Aufruf definiert sein
- Funktionen werden dazu verwendet den Code zu unterteilen und nachträgliches Ändern leichter zu machen
- Eine Funktionsdefinition erfolgt über das Schlüsselwort *def* + *Funktionsname* + ()

```
def helloWorld():  
    print("Hello World!")  
  
helloWorld()
```

MODULE UND WEBANFRAGEN

MODULE

- Die Python Community stellt ihren Code in Form von Modulen zur Verfügung z.B. NumPy
- Module werden mit dem Schlüsselwort ***import*** hinzugefügt
- Um nur Teile eines Modules zu importieren, kann ***from ... import ...*** verwendet werden

```
#!/usr/bin/python

import numpy

import tensorflow as tf

from urllib import request
```

MODULE UND WEBANFRAGEN

WEBANFRAGEN

- Für Webanfragen muss das Module **requests** importiert werden, welches mehrere Funktionen enthält:
 - get()
 - status_code()
 - headers()
 - encoding()
 - text(), etc.

```
#!/usr/bin/python

import requests

website = requests.get('https://www.th-deg.de/')
print(website.status_code)
print(website.text)
```

PYTHON NETWORK SOCKETS

PYTHON CLIENTS

- Netzwerk Sockets sind Endpunkte eines Netzwerks zum Senden und Empfangen von Daten.
- Zum Verwenden von Netzwerk Sockets muss das Modul **socket** importiert werden.
- Vorgehensweise:
 1. Setzen einer Socket-Variable
 2. Verbinden zum Server
 3. Empfangen von Daten
 4. Senden von Daten
 5. Schließen der Verbindung

```
#!/usr/bin/python

import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

s.connect(("192.168.1.2", 1234))

print(s.recv(1024))

s.close()
```